Git Collaborator Access Manager

Complete Architecture, Implementation, API, Database, Security, Deployment, and Interview Guide

Repository: Github-Collabrator-Access-Manager
Branch inspected: testing
Analysis date: 29 September 2026
Audience: Fresher preparing for a Java backend developer interview

Evidence policy

Every implementation statement in this guide was checked against the repository. Paths and method names are supplied for traceability. “Implemented” means executable code or configuration exists. “Inferred” means behavior follows from code but was not observed against a live external system. “Missing” means the repository contains no implementation for the mechanism. Secret values are intentionally omitted.

Document map

Start with Sections 1-2 for an interview summary. Use Sections 3-8 to trace code and runtime behavior. Sections 9-12 cover risk, setup, and troubleshooting. Sections 13-15 are a study guide and implementation index.

Table of Contents

Table of Contents 2

1. Project Overview 4

- Major implemented features . . . 4
- User roles . . . 4
- Technology stack . . . 4
- 60-second interview explanation . . . 4

2. High-Level System Architecture 4

- How data travels . . . 5

3. Low-Level Architecture 5

- Backend controller map . . . 5
- Service responsibilities . . . 5
- Class relationship view . . . 6
- DTO and middleware reality . . . 6

4. Database Architecture 6

- Database catalog . . . 7
- ER diagram . . . 7
- CRUD traces . . . 8
- Representative JPQL . . . 8

5. Complete API Documentation 8

- Common headers and status behavior . . . 10
- Example: create employee . . . 10
- Example: sync . . . 10
- Important endpoint sequence . . . 10

6. Third-Party Integrations 10

7. Frontend Architecture and Complete UI Flow 11

- Component behavior . . . 11
- Loading, validation, and errors . . . 11

8. End-to-End Execution Flows 11

- Startup . . . 11
- Login . . . 12
- Sync . . . 12
- Map identity . . . 12
- Immediate employee removal . . . 12

Approval-gated bulk offboarding . . . 12
Failure behavior . . . 12
Sync sequence diagram . . . 12

9. Authentication and Security 12

Security sequence and gap . . . 13
Recommended improvements (not implemented) . . . 13

10. Dependencies, Packages, and Configuration 13

Environment variables (names only) . . . 13
Build process . . . 13

11. Docker, Infrastructure, and Deployment 14

Run locally . . . 14
Deployment findings . . . 14

12. Error Handling and Debugging 14

Troubleshooting checklist . . . 14

13. Interview Preparation 15

30-second version . . . 15
3-minute version . . . 15
Likely interview questions . . . 15

14. Implementation Traceability 15

Source location key . . . 16

15. Verification, Findings, and Glossary 16

Verification matrix . . . 16
Key verified gaps . . . 16
Glossary . . . 16
Final assessment . . . 16

1. Project Overview

Purpose

The application inventories repositories visible to a configured GitHub personal access token, records direct collaborator access, maps GitHub identities to employees, and removes access during offboarding. It adds operational records for synchronization, removal requests, bulk approvals, and audit history.

Major implemented features

- Admin login and logout using a signed cookie.
- Manual GitHub synchronization of repositories, collaborators, and effective permission labels.
- Repository and collaborator browsing, search, visibility filtering, and detail pages.
- Employee creation, editing, deletion, and GitHub-account mapping.
- Single collaborator removal, employee-wide removal, explicit removal requests, offboarding jobs, and approval-gated bulk offboarding.
- Audit log and sync-status views.
- Docker Compose and Render deployment definitions.

User roles

The token payload type permits ADMIN and VIEWER, but login always issues ADMIN and no code enforces role-specific permissions. The practical implementation is a single configured administrator account. This is an important interview distinction: role labels exist, role-based authorization does not.

Technology stack

Layer	Verified technology
Browser UI	Next.js 15 App Router, React 19, TypeScript 5.6, Tailwind CSS 3.4
Web/API gateway	Next.js route handlers and middleware
Backend	Java 23, Spring Boot 3.4.1, Spring Web, Spring Data JPA, Hibernate
Data	PostgreSQL 16; Prisma 5 schema/client in frontend; Hibernate ddl-auto=update in backend
External	GitHub REST API version 2022-11-28 using a bearer PAT
Runtime	Node 20 Alpine, Eclipse Temurin 23, Docker Compose; optional Nginx 1.27; Render blueprint

60-second interview explanation

> “I built a GitHub collaborator access manager with a Next.js frontend and a Spring Boot backend. An admin signs in, runs a GitHub sync, and the backend calls the GitHub REST API to import repositories, collaborators, and permissions into PostgreSQL. HR-style employee records can be mapped to GitHub identities. The offboarding workflows remove access through GitHub, then update local access rows and write audit records. Spring Data JPA manages backend persistence, while Prisma supplies a matching schema and types to the Next.js layer. Docker Compose runs PostgreSQL, Spring Boot, and Next.js; a production overlay adds Nginx. I would improve it by enforcing session validation in middleware and Spring endpoints, adopting one migration owner, and adding centralized exception handling.”

2. High-Level System Architecture

Verified runtime topology

```
Browser
  | HTTPS/HTTP: pages, forms, fetch(/api/*), cookie gcam_session
  v
Nginx (production overlay only, port 80)
  | reverse proxy
  v
Next.js 15 (port 3000)
  |-- Server Components render pages
  |-- Middleware checks cookie presence
  |-- Route Handlers expose same-origin /api/*
  |       | HTTP JSON via SPRING_BOOT_URL
```

```
  |       v
  |    Spring Boot 3.4.1 (port 8080)
  |       | JPA/JDBC
  |       +----------------------> PostgreSQL 16 (5432 container)
  |       | HTTPS + bearer PAT
  |       +----------------------> api.github.com
  |
  +-- Prisma Client ----------------> PostgreSQL (limited direct use: schema sync,
                                      bootstrap, repository permission precheck)
```

How data travels

A user action normally calls a relative Next.js /api route. That handler validates basic input and calls a TypeScript service. The service uses fetchBackend to send JSON to Spring Boot. A controller delegates to a service, which uses a Spring Data repository and may call GitHub. The response returns through Spring, the Next route, and the browser; client components then show a message and call router.refresh(), causing server components to fetch current data again.

> Architectural nuance

The repository retains two server-side GitHub clients: GitHubClientService.java in Spring and services/github/client.ts in Next.js. Normal sync and removals use Spring. The Next client is still used by the Next repository-removal route to pre-check admin permission and prevent self-removal before Spring repeats the permission check. This duplicates external calls and policy logic.

3. Low-Level Architecture

Actual source organization

```
backend/
  pom.xml, Dockerfile
  src/main/java/com/gitcollab/manager/
    controller/  9 REST controllers
    service/     business workflows, GitHub client, transaction boundaries
    repository/  12 Spring Data JPA repositories
    model/       12 entities plus 3 enums
  src/main/resources/application.properties
  src/test/.../OnboardingWorkflowIntegrationTest.java
git-collabration/
  src/app/       App Router pages and 16 route-handler files
  src/components client/server UI components
  src/services/  typed Spring API adapters; second GitHub client
  src/lib/       auth, env, backend fetch, Prisma, date, JSON helpers
  prisma/schema.prisma, prisma/seed.ts
  Dockerfile, docker-entrypoint.sh
nginx/nginx.conf; docker-compose*.yml; render.yaml; docs/
```

Backend controller map

Controller	Responsibilities	Primary dependencies
AuthController	login, logout	SessionManager
DashboardController	overview	DashboardService
EmployeeController	employee CRUD; account mapping	EmployeeService
CollaboratorController	search; remove from all repos	CollaboratorService, OffboardingService
RepositoryController	browse, metrics, access, detail, bulk collaborator removal	RepositoryService, GitHubClientService, OffboardingService
SyncController	run/list sync jobs	SyncService
OffboardingController	jobs, direct removal, removal requests, bulk batches	OffboardingService
GovernanceController	approvals and audit logs	GovernanceService, AuditLogService
HealthController	liveness JSON	none

Service responsibilities

Service	Responsibility
SyncService	coordinates full sync and job state
SyncTransactionService	REQUIRES_NEW upsert per repository
OffboardingService	creates/executes all removal workflows
OffboardingTransactionService	REQUIRES_NEW local deletion and identity detach
EmployeeService	CRUD and identity mapping
RepositoryService	repository queries and metrics
GovernanceService	approval transitions and bulk execution
DashboardService	counts and last sync with fallback
GitHubClientService	GitHub REST, pagination, one rate-limit retry
SessionManager	HMAC-SHA256 token creation/verification
AuditLogService	serializes and persists event detail
CollaboratorService	search facade

Class relationship view

```
Controller -> Service -> JpaRepository<T, UUID> -> PostgreSQL
                    |
                    +-> GitHubClientService -> GitHub REST API

Employee 1 ---- * GithubUser * ---- * Repository
                        via RepositoryAccess
Employee 1 ---- * OffboardingJob 1 ---- * OffboardingJobItem ---- 1 Repository
ApprovalRequest 1 ---- 0..1 BulkOffboardingBatch 1 ---- * BatchItem ---- 1 Employee
GithubUser 1 ---- * RemovalRequest * ---- 1 Repository
SyncJob 1 ---- * SyncJobDetail * ---- 0..1 Repository
```

DTO and middleware reality

There is no backend DTO package. Controllers accept Map<String, String> or Map<String, Object> and return entities/maps directly. Validation annotations such as @Valid are not used. Next middleware protects all non-public page and API paths by checking only whether gcam_session exists. PageShell independently calls getSession(), which verifies the HMAC, and redirects invalid sessions. API route handlers do not call getSession().

4. Database Architecture

PostgreSQL is the verified database. Prisma schema.prisma defines the complete relational model used by the Next build/runtime, while JPA entities map the same tables. No Prisma migrations directory or Flyway/Liquibase scripts exist. The frontend entrypoint runs prisma db push; Spring uses spring.jpa.hibernate.ddl-auto=update. Therefore schema evolution has two owners and is not versioned as migrations.

Database catalog

Table	Columns and types (logical)	Constraints / relationships
employees	id UUID PK; full_name text; email nullable; department nullable; is_active bool default true; notes nullable; created_at; updated_at	index full_name
github_users	id UUID PK; github_id bigint nullable; github_username text; github_profile_url nullable; employee_id UUID nullable FK; timestamps	unique github_username; index username; employee FK SET NULL
repositories	id UUID PK; github_repo_id bigint; name; full_name; description; live_url; visibility; default_branch; archived; is_active; last_synced_at; timestamps	unique github_repo_id; index name
repository_access	id UUID PK; repository_id UUID FK; github_user_id UUID FK; permission; source; synced_at; timestamps	unique(repo,user); both FKs indexed and CASCADE
sync_jobs	id UUID PK; started_at; completed_at; status; initiated_by; repositories_scanned; collaborators_found; error_count; error_message; created_at	status enum in Java; string in Prisma
sync_job_details	id UUID PK; sync_job_id FK; repository_id FK; status; error_message; created_at	job CASCADE; repository SET NULL
audit_logs	id UUID PK; action_type; entity_type; entity_id UUID; performed_by; details JSON; created_at	entity_id is logical reference, not FK
removal_requests	id UUID PK; github_user_id FK; repository_id FK; status; requested_by; reason; executed_at; created_at	both FKs SET NULL
offboarding_jobs	id UUID PK; employee_id FK; status; totals; processed; failed; started_at; completed_at; created_at	employee SET NULL
offboarding_job_items	id UUID PK; job_id FK; repository_id FK; status; error_message; created_at	job CASCADE; repository SET NULL
approval_requests	id UUID PK; request_type; reference_id; status; requested_by; approved_by; approved_at; created_at	reference_id is text, not FK
bulk_offboarding_batches	id UUID PK; approval_request_id FK; requested_by; reason; status; executed_by; approved_at; executed_at; rejection_reason; timestamps	approval_request_id unique, SET NULL
bulk_offboarding_batch_items	id UUID PK; batch_id FK; employee_id FK; offboarding_job_id FK; status; error_message; timestamps	job id unique; batch/employee CASCADE; job SET NULL

ER diagram

```
employees ||--o{ github_users : maps
github_users ||--o{ repository_access : has
repositories ||--o{ repository_access : grants
sync_jobs ||--o{ sync_job_details : contains
repositories |o--o{ sync_job_details : reports
github_users |o--o{ removal_requests : targets
repositories |o--o{ removal_requests : targets
employees |o--o{ offboarding_jobs : subject
offboarding_jobs ||--o{ offboarding_job_items : contains
repositories |o--o{ offboarding_job_items : processed
approval_requests |o--o| bulk_offboarding_batches : gates
bulk_offboarding_batches ||--o{ bulk_offboarding_batch_items : contains
employees ||--o{ bulk_offboarding_batch_items : subjects
offboarding_jobs |o--o| bulk_offboarding_batch_items : result
```

CRUD traces

Operation	Verified code path and database action
Insert employee	EmployeeForm -> POST /api/employees -> EmployeeController.createEmployee -> EmployeeService.createEmployee -> EmployeeRepository.save
Update employee	employee detail form server action -> EmployeeService update route path; Spring updateEmployee -> findById, uniqueness check, save
Fetch repositories	Repository page -> listRepositories -> Next /api/repositories -> Spring RepositoryRepository.searchRepositories JPQL
Delete employee	DELETE /api/employees/{id} -> EmployeeRepository.deleteById; behavior depends on database relations
Sync replace access	SyncTransactionService saves repository, deletes all access rows for it, then inserts current collaborator access rows
Remove access	GitHub DELETE first; OffboardingTransactionService then deletes RepositoryAccess and unmaps a user only when no access remains

Representative JPQL

```
SELECT r FROM Repository r WHERE (:search = '' OR LOWER(r.name) LIKE LOWER(CONCAT('%', :search, '%')) OR LOWER(r.fullName) LIKE LOWER(CONCAT('%
```

```
DELETE FROM RepositoryAccess ra WHERE ra.repository.id = :repositoryId AND ra.githubUser.id = :githubUserId
```

5. Complete API Documentation

Method	URL	Input	Success response	Auth / behavior
POST	/api/auth/login	form: username,password,from	302 + signed cookie	public; Next and Spring versions exist
POST	/api/auth/logout	none	302 /login; expire cookie	public
GET	/api/health	none	{ok,service}	public
GET	/api/dashboard/overview	none	counts,lastSyncJob,DB state	Spring endpoint; Next page calls service directly
GET	/api/employees	q	{employees:[...]}	cookie presence via Next middleware
POST	/api/employees	{fullName,email}	201 {employee}	fullName required; duplicate email -> 400
GET	/api/employees/{id}	UUID path	employee or 404	Spring endpoint; no Next route file
PATCH	/api/employees/{id}	partial employee JSON	{employee} or 400	Spring endpoint; no Next route file
DELETE	/api/employees/{id}	UUID path	{success:true} or 400	Spring returns wrapper; TS service type is mismatched
POST	/api/github-users/mapping	{githubUserId,employeeId?}	{githubUser}	empty employeeId unmaps
GET	/api/collaborators	q	{collaborators:[...]}	only users with access
POST	/api/collaborators/remove-all	{githubUserId,executedBy?}	{result}	GitHub DELETE for each access
GET	/api/repositories	q,visibility	{repositories:[...]}	visibility all/public/private
GET	/api/repositories/metrics	none	counts,lastSyncJob	Spring only
GET	/api/repositories/{id}	UUID or numeric GitHub id	repository or 404	Spring only
GET	/api/repositories/{id}/access	UUID	access array	Spring only
POST	/api/repositories/remove-collaborators	{repositoryId,githubUserIds[],executedBy?}	{result}; 403 if no admin	Next and Spring both verify GitHub admin
GET	/api/sync	none	latest 20 jobs	Spring only
POST	/api/sync	{initiatedBy?}	sync result	rejects concurrent RUNNING job
GET	/api/offboarding/jobs	none	{offboardingJobs}	latest 50
POST	/api/offboarding/jobs	{employeeId,requestedBy?,reason?}	201 {offboardingJob}	creates job/items/removal requests
POST	/api/offboarding/jobs/{id}/execute	{executedBy?}	{offboardingJob}	GitHub removals + local updates
POST	/api/offboarding/remove	{employeeId,executedBy?}	{result}	immediate employee-wide removal
GET	/api/removal-requests	none	{removalRequests}	latest 50
POST	/api/removal-requests	userId,repositoryId,metadata	201 {removalRequest}	validates both identifiers
POST	/api/removal-requests/{id}/execute	{executedBy?}	{removalRequest}	idempotent when EXECUTED
GET	/api/offboarding/bulk	none	{bulkBatches}	latest 50
POST	/api/offboarding/bulk	employeeIds[],requestedBy?,reason?	201 {bulkBatch}	also creates approval request
GET	/api/governance/approvals	none	{approvalRequests}	latest 100
POST	/api/governance/approvals	requestType,referenceId?,requestedBy?	201 {approvalRequest}	requestType required
PATCH	/api/governance/approvals/{id}	status APPROVED/REJECTED, actedBy?	{approvalRequest}	approval triggers bulk execution
GET	/api/governance/audit-logs	none	{auditLogs}	latest 100
POST	/api/governance/audit-logs	action/entity/detail JSON	201 audit log	Spring endpoint used internally by Next employee/mapping routes

Common headers and status behavior

Browser requests use Content-Type: application/json except login forms. Next fetchBackend always adds JSON content type. GitHub calls add Accept: application/vnd.github+json, Authorization: Bearer [redacted], X-GitHub-Api-Version: 2022-11-28, and User-Agent. Controllers mostly translate any caught exception to HTTP 400. There is no global exception handler, formal error envelope, OpenAPI definition, or @Valid DTO validation.

Example: create employee

```
POST /api/employees
Content-Type: application/json

{"fullName":"Asha Verma","email":"asha@example.test"}

HTTP 201
{"employee":{"id":"11111111-1111-1111-1111-111111111111",
 "fullName":"Asha Verma","email":"asha@example.test","isActive":true}}

HTTP 400 example
{"error":"An employee with this email already exists."}
```

Example: sync

```
POST /api/sync
{"initiatedBy":"admin"}

HTTP 200
{"success":true,"repositoriesProcessed":12,"collaboratorsProcessed":34,
 "errors":0,"status":"SUCCESS"}

Failure still returns HTTP 200 when initial GitHub repository fetch fails:
{"success":false,"repositoriesProcessed":0,"collaboratorsProcessed":0,
 "errors":1,"status":"FAILED","errorMessage":"GitHub repository fetching failed: ..."}
```

Important endpoint sequence

```
Browser -> Next POST /api/repositories/remove-collaborators
Next route -> Prisma: find repository full_name
Next route -> GitHub: GET repository permissions + GET /user
Next route -> Spring: POST same endpoint
Spring -> PostgreSQL: load repository and target users
Spring -> GitHub: repeats permission check and self-removal check
Spring -> GitHub: DELETE collaborator(s)
Spring -> PostgreSQL: delete access; possibly detach employee mapping; add audit log
Spring -> Next -> Browser: per-user result; router.refresh()
```

6. Third-Party Integrations

Integration	Purpose and authentication	Calls	Resilience / limits
GitHub REST API	PAT from GITHUB_PAT; bearer header	GET /user/repos; GET repo collaborators; GET collaborator permission; GET repo; GET /user; DELETE collaborator	10s connect timeout in Java; no request timeout; pagination 100/page; one retry only when 403 and remaining=0; waits until reset; no general backoff
PostgreSQL	Credentials from datasource variables / DATABASE_URL	JPA and Prisma SQL generated by ORMs	Docker healthcheck; frontend schema push retries 15 times; no application-level DB retry
Render	render.yaml blueprint	managed PostgreSQL + two Docker web services	health paths configured; secrets grouped; no CI pipeline
Nginx	optional local/VPS production proxy	HTTP reverse proxy to Next	90s proxy read timeout, gzip, static caching, basic security headers

Not implemented

- GitHub OAuth or GitHub App installation flow.
- Webhook ingestion; synchronization is manual.
- Automated rate-limit telemetry, exponential backoff, circuit breaker, or request queue.
- Cloud object storage, email, Slack, queues, or scheduled jobs.
- A use of GITHUB_OWNER in the executed GitHub API flow; repository discovery uses /user/repos.

7. Frontend Architecture and Complete UI Flow

Route	Verified purpose
/	Dashboard counts, last sync, navigation and SyncButton
/login	Admin credential form; preserves from redirect
/repositories	RepositoryBrowser with local search and visibility filter
/repositories/[id]	Repository details; permission check; selectable collaborator removal
/collaborators	Search and collaborator cards
/collaborators/[id]	Redirects to collaborators list
/employees	Employee creation plus client-side search/list
/employees/[id]	Edit employee, view mappings/access, remove employee access
/mapping	MappingFocusSection / MappingForm
/sync-status	Latest sync jobs
/offboarding	Direct and bulk offboarding consoles plus job/request history
/governance	Approval console and audit history

Component behavior

Component	Major action and UI state
SyncButton	POST /api/sync; loading; success/error toast; gcam_sync_done cookie; refresh
EmployeeForm	POST /api/employees; status; navigate/refresh
MappingForm	POST /api/github-users/mapping; disabled unless choices exist; refresh
RepositoryBrowser	client-side search/visibility; links to detail/live URL
RepositoryCollaboratorsList	selection, confirmation dialog, POST bulk removal, result toast
RemoveCollaboratorButton	confirm, POST remove-all, refresh
OffboardingRemovalConsole	search/select employee, preview repos, immediate removal
BulkOffboardingConsole	multi-select employees, reason, creates approval-gated batch
GovernanceConsole	create request; approve/reject; refresh
DialogProvider	promise-based confirm/alert modal with keyboard handling
SiteNav/PageShell	navigation, verified-session shell, logout
Toast/StatCard/CopyButton	presentation and feedback helpers

UI navigation

```
Login -> Dashboard
  |-> Repositories -> Repository detail -> remove selected collaborators
  |-> Collaborators -> remove one identity from every repository
  |-> Employees -> Employee detail -> edit / remove all access
  |-> Mapping -> map GitHub identity to employee
  |-> Sync Status
  |-> Offboarding -> direct removal / bulk request / job history
  +-> Governance -> approve/reject -> bulk execution + audit log
```

Loading, validation, and errors

Client components use local state flags such as loading, isRemoving, or isSubmitting, disable controls, display inline status/toasts, and refresh server-rendered data after success. Confirmation dialogs protect destructive UI actions. Validation is mostly required-field checks in the route handler/controller; malformed UUIDs and service errors become 400 responses. Several read services catch backend failures and return empty arrays, which can make an outage look like an empty dataset.

8. End-to-End Execution Flows

Startup

Docker starts PostgreSQL -> healthcheck -> Spring -> healthcheck -> Next. Next entrypoint runs prisma db push with retries, then npm start. Spring Hibernate also performs ddl-auto=update.

Login

POST form to Next /api/auth/login -> compare ADMIN_USERNAME/PASSWORD -> HMAC token payload {username,role:ADMIN} -> HttpOnly, SameSite=Lax, Secure in production cookie -> redirect. No expiry exists inside token; cookie has session lifetime in Next and 24h in Spring variant.

Sync

UI -> Next -> Spring SyncService -> create RUNNING SyncJob -> GitHub paginated repos -> for each repo collaborators and permission calls -> isolated transaction upserts repository/users and replaces access -> inactive missing repos -> finalize job + audit -> refresh UI.

Map identity

MappingForm -> Next handler -> Spring EmployeeService -> load GitHub user and optional employee -> set relation -> save -> Next separately records audit event.

Immediate employee removal

UI confirms -> Spring collects every mapped GitHub identity/repository pair -> GitHub DELETE each -> isolated DB deletion -> detach identity when no access remains -> audit summary -> UI result.

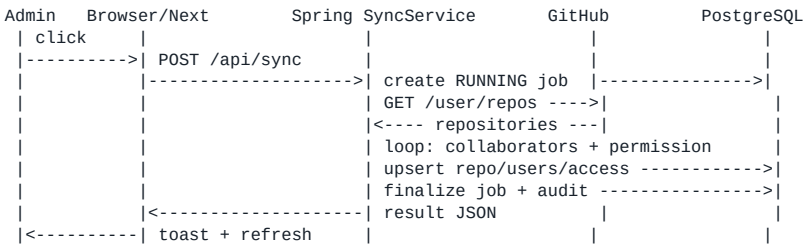
Approval-gated bulk offboarding

Create batch and items -> create PENDING approval -> governance APPROVE -> mark batch approved -> synchronously create and execute one offboarding job per employee -> finalize item/batch statuses -> audit.

Failure behavior

Sync records FAILED or PARTIAL_SUCCESS. Offboarding treats a GitHub “not found” message as already removed; other errors mark item/request failed. Transactions isolate each repository/access deletion, but external GitHub deletion and local DB update are not one atomic transaction.

Sync sequence diagram



9. Authentication and Security

Status	Finding
Implemented	HMAC-SHA256 signed session token; HttpOnly cookie; SameSite=Lax in Next; Secure in production Next; timing-safe comparison in Next verifier; GitHub PAT stays server-side; parameterized ORM queries; non-root Docker users; Nginx headers; repository admin/self-removal checks.
Partially implemented	Middleware checks cookie presence but not its signature. PageShell verifies it for rendered pages. Spring SessionManager can verify tokens, but no filter/interceptor calls it. Role is stored but never authorized. CORS is absent because browser normally calls same-origin Next routes; direct Spring exposure remains possible.
Missing	Spring Security; endpoint-level authentication; CSRF tokens; rate limiting; session expiry/revocation; account lockout; password hashing/user table; request DTO validation; centralized sanitization; CSP/HSTS; secret-strength enforcement; audit tamper protection.

Security sequence and gap

```
Request -> Next middleware
  cookie absent: redirect /login
  cookie present (any value): allow request
    page render -> PageShell -> verify HMAC -> redirect if invalid
    /api route -> no session verification -> handler can run
Spring /api/* -> no authentication filter -> controller can run if directly reachable

Consequence: authentication protects the normal UI path incompletely. Treat this as a
verified implementation gap, not as production-grade access control.
```

Recommended improvements (not implemented)

- Validate the signed session in middleware and every Next route, or use a standard session library.
- Protect Spring endpoints with Spring Security and pass/validate an identity token from Next.
- Add expiration, rotation, logout revocation strategy, role authorization, and CSRF protection.
- Replace default credentials/secrets; reject weak production values; set Secure consistently behind TLS.
- Use DTOs with Bean Validation and a @RestControllerAdvice error envelope.
- Avoid returning raw GitHub error bodies because they may reveal operational details.

10. Dependencies, Packages, and Configuration

Area	Dependency/version	Verified purpose
Backend	Spring Boot parent 3.4.1	dependency management and runtime
Backend	spring-boot-starter-web	REST/MVC and embedded server
Backend	spring-boot-starter-data-jpa	JPA/Hibernate repositories/transactions
Backend	spring-boot-starter-validation	present, but Bean Validation DTOs are not used
Backend	postgresql runtime	JDBC driver
Backend	spring-boot-starter-test	JUnit/Spring/Mockito tests
Frontend	next ^15.0.0	App Router, server rendering, route handlers
Frontend	react/react-dom ^19.0.0	UI
Frontend	@prisma/client/prisma ^5.0.0	schema, generated types/client, db push
Frontend	zod ^3.23.8	environment parsing
Frontend	clsx ^2.1.1	conditional CSS classes
Frontend	tailwindcss ^3.4.13	styling
Tooling	TypeScript ^5.6.3; tsx ^4.19.1	typing and seed execution

Environment variables (names only)

Name	Use
DATABASE_URL	Prisma PostgreSQL URL
SPRING_DATASOURCE_URL/HOST/PORT/DATABASE/USERNAME/PASSWORD	Spring JDBC connection
POSTGRES_DB/USER/PASSWORD/PORT	Compose database initialization and host mapping
SPRING_BOOT_URL	Next server-to-Spring base URL
GITHUB_PAT	server-side GitHub bearer token
GITHUB_OWNER	configured, but unused by executed GitHub discovery
SESSION_SECRET	HMAC signing secret
ADMIN_USERNAME / ADMIN_PASSWORD	single admin credential
SPRING_BOOT_PORT / FRONTEND_PORT / HTTP_PORT	host port mappings
JAVA_OPTS	JVM container memory settings
PORT / HOSTNAME / NODE_ENV / NEXT_TELEMETRY_DISABLED	runtime settings

Build process

Frontend prebuild runs scripts/sync-schema.mjs; build runs prisma generate and next build. Its Dockerfile repeats Prisma generation, then copies the built .next directory and dependencies into a non-root runtime. Backend uses a Maven build stage and copies the JAR into a Temurin JRE runtime. Backend Docker packaging skips tests.

11. Docker, Infrastructure, and Deployment

```
Development / base Compose
Host 5433 -> postgres:5432 (volume postgres_data)
Host 8080 -> backend:8080 -> postgres:5432 + api.github.com:443
Host 3000 -> frontend:3000 -> backend:8080 + postgres:5432
All services: gcam-network bridge; healthchecks; JSON log rotation 10 MB x 3

Production overlay
Host 80 -> nginx:80 -> frontend:3000

Render blueprint
Managed PostgreSQL -> gcam-backend (Docker) -> gcam-frontend (Docker)
Frontend SPRING_BOOT_URL points to the public backend URL.
```

Run locally

```
# Configure environment variables without committing secrets.
docker compose up -d --build
docker compose ps

# With Nginx
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Deployment findings

- PostgreSQL data persists in the named postgres_data volume.
- Backend and frontend containers run as non-root users.
- The database port and backend port are published by the base Compose file, even when Nginx is added.
- No GitHub Actions or other CI/CD workflow was found. Render blueprint deployment configuration is present.
- The frontend README mentions Vercel and Neon, but current root infrastructure also defines Docker/Render. Treat Vercel/Neon as documentation intent, not the only verified deployment.

12. Error Handling and Debugging

Area	Actual behavior
Validation	manual required-field checks; most errors -> {error: message}, HTTP 400
Authentication	invalid credentials redirect to /login?error=invalid; invalid page token redirects via PageShell
Database	dashboard returns zero counts and a notice; many read services return empty arrays; writes surface 400
GitHub	status >=400 throws with response body; one rate-limit retry; sync records job failure; removal tracks partial failure
Transactions	sync and access deletion use REQUIRES_NEW for per-unit isolation
Logging	Spring DEBUG package logging and show-sql=true; System.out/err; Next console.error; Docker log rotation
Global handling	no @ControllerAdvice, no typed shared error contract, no correlation IDs

Troubleshooting checklist

- Empty dashboards: check Spring /api/health, PostgreSQL health, SPRING_BOOT_URL, and datasource variables.
- Sync fails immediately: verify GITHUB_PAT exists and its GitHub scopes can list repositories/collaborators.
- Removal returns 403: token must report permissions.admin=true for that repository.
- Stale RUNNING job: Next bootstrap can mark it FAILED after restart, but this helper must be invoked by application code; verify its call path before relying on it.
- Schema mismatch: compare JPA entities and schema.prisma; both automatic schema mechanisms are active.
- Render cold start: frontend catches backend fetch failures and may show empty lists; refresh after both services are healthy.

13. Interview Preparation

30-second version

This project centralizes GitHub repository access. Spring Boot syncs data from GitHub into PostgreSQL, Next.js presents dashboards and workflows, and administrators map GitHub users to employees and remove access during offboarding. The design records jobs, approvals, and audit events and runs with Docker.

3-minute version

The browser talks to Next.js, which provides both server-rendered pages and same-origin API routes. Those route handlers call Spring Boot over HTTP. Spring controllers are thin; services contain sync, employee, governance, and offboarding rules; Spring Data repositories persist JPA entities in PostgreSQL. During sync, the backend pages through repositories and collaborators from GitHub, asks GitHub for each permission, then uses an isolated transaction per repository to upsert the repository and GitHub users and replace access rows. Employee mappings add the human identity layer. Removal workflows call GitHub first and only then remove local access, recording partial failures and audit events. Bulk offboarding creates a governance approval; approval synchronously executes an offboarding job per employee. Docker Compose coordinates PostgreSQL, backend, and frontend, and an optional Nginx proxy. The strongest design ideas are explicit workflow records and per-unit transactions. The main production gaps are incomplete authentication enforcement, no versioned migrations, duplicated GitHub policy calls, weak default secrets, and inconsistent error handling.

Likely interview questions

Question	Grounded answer
Why Spring Boot?	It provides REST, dependency injection, JPA repositories, and transaction control with little infrastructure code.
Why RepositoryAccess?	It models the many-to-many relationship and stores permission, source, and sync time.
Why REQUIRES_NEW?	One repository or removal can commit/fail independently, enabling partial success.
How is consistency handled with GitHub?	Remote deletion happens before local deletion. Failures are recorded, but there is no distributed transaction or compensation.
How does sync avoid stale rows?	It deletes and recreates access rows per repository and marks repositories absent from the current fetch inactive.
What is the N+1 risk?	Sync requests each collaborator permission separately; entity graphs may also trigger many relationship queries.
How would you scale sync?	Queue repository tasks, cap concurrency, batch permission retrieval where possible, checkpoint jobs, and respect rate limits.
Is it secure?	It has signed cookies and server-side PATs, but API authentication and Spring Security are missing; it needs hardening.
How are schemas migrated?	Currently Hibernate update and Prisma db push. A production design should choose one owner and committed migrations.
How are failures exposed?	Jobs and items store statuses/errors; most API exceptions become 400; read failures may become empty data.
What does approval do?	For BULK_OFFBOARDING it changes batch state and immediately executes the batch synchronously.
Where is RBAC?	A role string exists, but no authorization rule uses it.
Why two ORMs?	Spring uses JPA for the backend; Prisma defines/types the shared schema and is used in a few Next operations. This increases drift risk.
How are duplicates prevented?	Unique GitHub username/repo ID and unique repository-user access; employee email uniqueness is application-enforced only.
What tests exist?	One Spring integration test covers health, sync-to-offboarding, bulk approval, and permission validation with a mocked GitHub client.
What would you improve first?	Enforce auth end-to-end, adopt migrations, centralize errors/DTO validation, then remove duplicated integration logic.
Is GITHUB_OWNER used?	It is configured but repository discovery calls /user/repos, so the verified runtime does not filter by owner.
How are rate limits handled?	A single retry waits until the reset timestamp only for 403 with zero remaining.
What is the API style?	REST-like JSON resources plus action endpoints such as /execute and /remove-all.
How is audit detail stored?	AuditLogService serializes a details map to PostgreSQL JSON through Hibernate.

14. Implementation Traceability

Feature	Frontend source	Backend source	Data / external
Login	app/login/page.tsx; api/auth/login/route.ts	AuthController.login; SessionManager	ADMIN_*; SESSION_SECRET
Dashboard	app/page.tsx; dashboard-service.ts	DashboardController.getOverview; DashboardService	Repository/GithubUser/SyncJob repos
Sync	components/sync-button.tsx; api/sync/route.ts	SyncController.runSync; SyncService; SyncTransactionService	repositories, users, access, sync jobs; GitHub
Repository browse	app/repositories/page.tsx; repository-browser.tsx	RepositoryController.getRepositories; RepositoryService	RepositoryRepository.searchRepositories
Repository removal	repository-collaborators-list.tsx; api/repositories/remove-collaborators	RepositoryController.removeCollaborators; OffboardingService	GitHub DELETE; repository_access; audit_logs
Collaborator removal	remove-collaborator-button.tsx; api/collaborators/remove-all	CollaboratorController.removeAllRepos	GitHub DELETE; repository_access
Employee CRUD	employee-form.tsx; app/employees/[id]/page.tsx	EmployeeController; EmployeeService	employees
Identity mapping	mapping-form.tsx; api/github-users/mapping	EmployeeController.mapGithubUser	github_users.employee_id; audit
Direct offboarding	offboarding-removal-console.tsx	OffboardingController.removeEmployeeAccess; OffboardingService	GitHub; repository_access; audit
Offboarding job	api/offboarding/jobs and execute	OffboardingController; OffboardingService	jobs/items/removal_requests
Bulk offboarding	bulk-offboarding-console.tsx; api/offboarding/bulk	OffboardingController; GovernanceService	batches/items/approval/jobs
Governance	governance-console.tsx; governance routes	GovernanceController; GovernanceService	approval_requests; audit_logs

Source location key

Frontend paths are under git-collabration/src/. Backend Java paths are under backend/src/main/java/com/gitcollab/manager/. Database definition: git-collabration/prisma/schema.prisma. Runtime configuration: backend/src/main/resources/application.properties, docker-compose.yml, docker-compose.prod.yml, render.yaml, and nginx/nginx.conf.

15. Verification, Findings, and Glossary

Verification matrix

Area	Evidence checked	Conclusion
APIs	all Spring controllers and all Next route.ts files	33 distinct Spring endpoints; Next exposes/proxies the browser-facing subset
Database	all JPA entities/repositories plus Prisma schema	13 tables; relationships and constraints documented
UI	all page.tsx files and component event/fetch handlers	12 page routes and major click/submit flows documented
Integration	Java and TS GitHub clients	actual endpoints, headers, pagination, retry, deletion documented
Security	auth helpers, middleware, controller/session code	controls and gaps separated
Infrastructure	Dockerfiles, Compose, Nginx, Render, READMEs	runtime paths and conflicting/stale hosting claims noted
Tests	OnboardingWorkflowIntegrationTest	four implemented integration scenarios; live GitHub not exercised

Key verified gaps

- No server-enforced authentication on Spring endpoints and no signature validation in Next middleware/API routes.
- No versioned database migration system despite two automatic schema synchronizers.
- No DTO layer or centralized exception handler.
- No CI/CD workflow in the repository.
- No GitHub OAuth/App/webhook implementation.
- No CSRF or application rate limiting.
- No direct automated frontend test suite was found.
- executeBulkOffboardingBatch in the TypeScript service is a stub returning success; actual bulk execution occurs inside Spring after approval.

Glossary

Term	Meaning in this project
Controller	HTTP entry point that parses a request and returns a response.
Service	Class containing business rules and workflow coordination.
Repository	Spring Data interface that reads/writes entities.
Entity	Java object mapped to a database table.
DTO	Purpose-built request/response object; this project mostly uses maps instead.
ORM	Object-relational mapper; JPA/Hibernate and Prisma translate objects to SQL.
Transaction	A unit of database work that commits or rolls back together.
HMAC	Secret-key signature used to detect token modification.
PAT	Personal access token sent to GitHub as a bearer credential.
RBAC	Role-based access control; role names exist here, enforcement does not.
CORS	Browser cross-origin policy; no explicit backend configuration exists.
CSRF	Forged browser request risk for cookie-authenticated apps; no explicit token control exists.
Idempotent	Safe to repeat without changing the final result; executed removal requests return unchanged.
N+1	Many extra calls/queries caused by looping over records.
Reverse proxy	Nginx receives client traffic and forwards it to Next.js.

Final assessment

The repository implements a substantial end-to-end access-management workflow and is particularly useful for discussing Spring service layering, JPA relations, external API coordination, and partial-failure handling. For a production deployment, authentication enforcement and database migration ownership are the first architectural risks to resolve.